Nile University

ECEN-438

Single Cycle vs Multi-Cycle vs Pipeline MIPS Processor

By:

John Samy Azer	ID: 18102206
Aser Hesham	ID: 18102527
Mustafa Abou Khatwa	ID: 18100598
Andrew Emad Samir	ID: 18101436
Salma Alaa Ibrahim	ID: 18100088

Abstract:

The purpose of this paper is to compare the performance of the single cycle, multicycle, and pipelining architecture using a simulated model. Through the use of Vivado Hlx 2016, we were able to design and construct all three processors using Verilog the programing language to achieve a suitable comparison. Each processor was designed to fulfill the designated requirements to emulate an actual processor in action to produce a proper comparison. Our findings were as follows, the single cycle was faster on short independent instructions beneath a certain number of instructions. The multicycle was faster on average than the single cycle architecture however it was still slower than the pipeline processor.

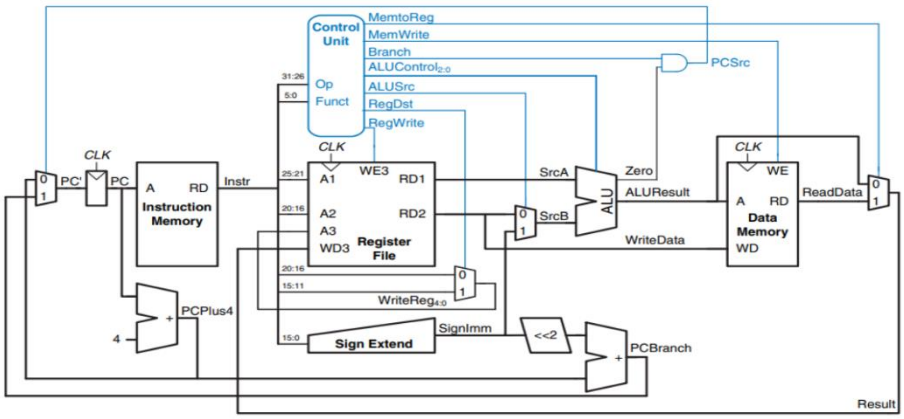
INTRODUCTION:

In our research project, we attempted to compare the performance of Single Cycle Processors, Multi-Cycle processors, and Pipelined processors to document the changes within each operation. To achieve this, we used Vivado HLx edition with a simulated General-Purpose Processor (GPP) as a programmable device that can run a wide range of applications. It's primarily intended for general-purpose devices like PCs and workstations. It benefits from a good time-to-market and flexibility. However, computation speed, power consumption, and hardware size are the main concerns for GPPs. The proposed processors in this paper are single-cycle MIPS and multi-cycle MIPS. This paper conducts a detailed comparison between the multi-cycle MIPS architecture and the single-cycle processor architecture in addition to the implementation of both processors using Verilog to show the main differences between them in terms of performance, execution time, and instruction execution cycle. In order to complete such a challenging task, an overview of the implementation of a multi-cycle MIPS processor, and a description of the data path and instruction cycle will be discussed. Following this, a more description of the code used in the simulation of the MIPS will be presented.

COMPARISON:

This section will show up the main differences between the two processors in terms of Block Diagram, advantages, disadvantages, data paths, and instruction execution cycle.

Single Cycle MIPS:



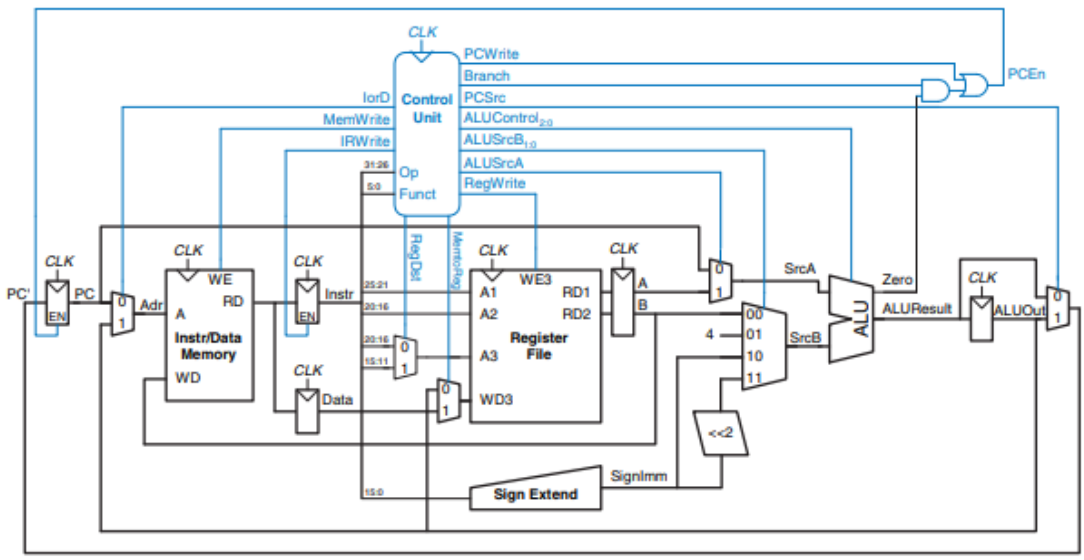
The main advantage of single cycle MIPS is the simplicity of design and implementation but on the other hand it is not used in computers anymore because its inefficient in performance and hardware cost, also clock cycle must have the same length for every instruction which is determined by slowest operation, so all instructions take the same amount of time regardless the actual processing time.

A MIPS instruction, which is always 32 bits wide, consists of the opcode and operands. The opcode describes the operation to be performed and the operands address the registers that hold the data. Instruction memory stores the instructions that will be executed. The register file contains the 32 MIPS general-purpose (integer/address) registers. The ALU performs all operations with ALUSrc control signal. In a single cycle, MIPS Instruction memory and data memory are separated. The data memory consists of only one Address input. MemRead and MemWrite control signals determine the actual memory operation. There are 2 adders for PC-based computations. Branch control signal used for branching to another instruction and multiplexers control signals to complete the desired operation successfully.

The instruction execution cycle is divided into five steps which are done in only one clock cycle:

1. Instruction Fetch: fetch the instruction and increment program counter.
2. Register Access: access registers data.
3. ALU operation: perform arithmetic or bitwise operations.
4. Memory Access: read or write in data memory.
5. Register access: write data to a specified register.

Multi-Cycle MIPS:



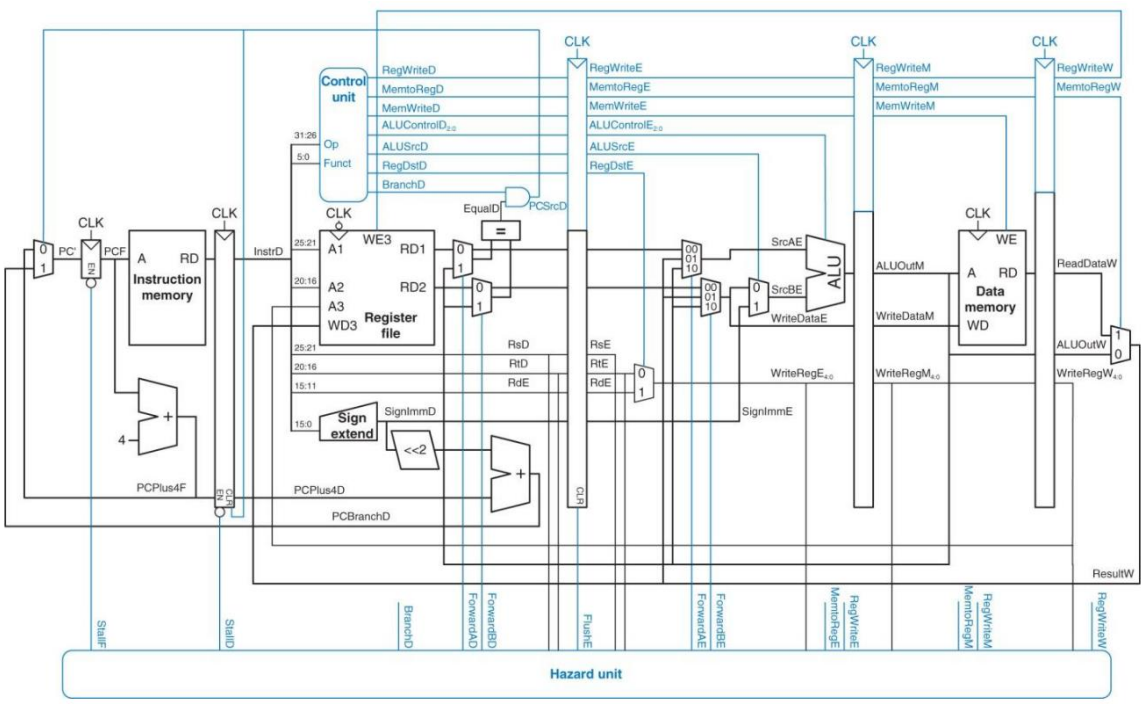
Multi-cycle MIPS reduces the overall hardware cost and allows units to be used more than once, also instruction is divided into steps to allocate a clock cycle for each step which increases the performance of the processor. ALU operations, register file access, and memory access occur in parallel leading to a decrease in the execution time.

Datapath of multi-cycle MIPS is similar to single-cycle MIPS in addition to a collection of functional units such as decoders, multipliers, and added multiplexers used in data processing operations. The main differences are the instruction memory, and the data memory is combined together in the same memory unit, a single ALU instead of three and registers added after each functional unit to hold the output for the next clock cycle.

The instruction execution cycle is divided into five steps each step is done in one clock cycle:

1. Instruction Fetch: fetch the instruction and increment program counter.
2. Instructions Decode and Register Fetch: read instruction opcode then read data from registers.
3. Execution, Memory Address Computation, or Branch Completion.
4. Memory Access or R-type Instruction Completion: The R-type instruction is completed, and results are stored in the appropriate register also, Memory access begins in this step in the case of I-type instruction.
5. Memory Read Completion: the data stored in the register file is either written to memory or is written to a specified register.

Pipeline MIPS:



In the pipeline processor, an instruction given will be processed in 5 stages each stage taking 1 clock cycle however unlike multicycle processors the pipeline processor can run multiple independent instructions simultaneously under the condition that between each independent instruction and another there is at least one idle cycle worth of time to function as a bubble to prevent the same resource

from being overused or misused by the instructions leading to hazards and physical damage to the hardware.

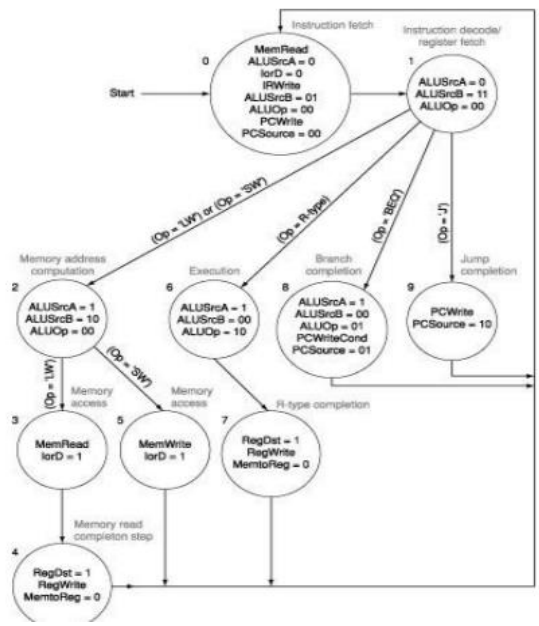
The data path of the pipeline processor is exactly like the single-cycle processor however 5 registers are added to separate the system into five stages. Each stage can work independently from the other due to the presence of registers. To remedy any possible conflicts due to dependencies of the instructions a hazard unit is added to mitigate any errors and possible harm to the system.

The instruction execution cycle is divided into five steps each step is done in one clock cycle:

1. Instruction Fetch: In this stage, the CPU reads instructions from the address in the memory whose value is present in the program counter.
2. Instruction Decode: In this stage, instruction is decoded, and the register file is accessed to get the values from the registers used in the instruction.
3. Instruction Execute: In this stage, ALU operations are performed.
4. Memory Access: In this stage, memory operands are read and written from/to the memory that is present in the instruction.
5. Write Back: In this stage, the computed/fetched value is written back to the register present in the instructions.

METHODOLOGY AND IMPLEMENTATION:

The implementation is done using Verilog on XILINX platform to obtain accurate results as possible. Each functional unit are implemented in a different module then all modules are called in testbench module. Modules are ALU, PC(Program counter), regfile (register file),inst_data_mem(Instruction/data memory),main decoder - maindec (Control unit), CLK_Register, current-state, next state,mux2x1, mux4x1 and test bench. All functional units start execution at the positive edge of the clock. The current state and next state are used to specify multi-cycle MIPS states as the following state finite machine. Instructions are stored in the program's visible state.



RESULTS AND DISCUSSION:

After our analysis on the matter, we constructed a theoretical environment where the three processors run the same set of codes where each set increases in complexity and dependency to test the outcome of each processor properly and accurately without unforeseen variables causing issues. By making a comparison between the three processors properly we can calculate the efficiency and comparison between each one the processors.

	Single Cycle	**Multi-Cycle**	**Parallel**
1 instruction	1 cycle	5 steps (equivalent to 1 cycle)	1 cycle
3 instructions with 1 to 2 dependencies	3 cycles	7 to 9 steps	1.4 to 1.8 single cycles worth of time
5 instructions with 2 to 4 dependencies	5 cycles	Over 10 steps	2.6 to 3.8 single cycles worth of time

We used a MIPS code consisting of 10 instructions of load/store and r-type instructions with some dependencies to compare the performance of the three architectures.

MIPS code:

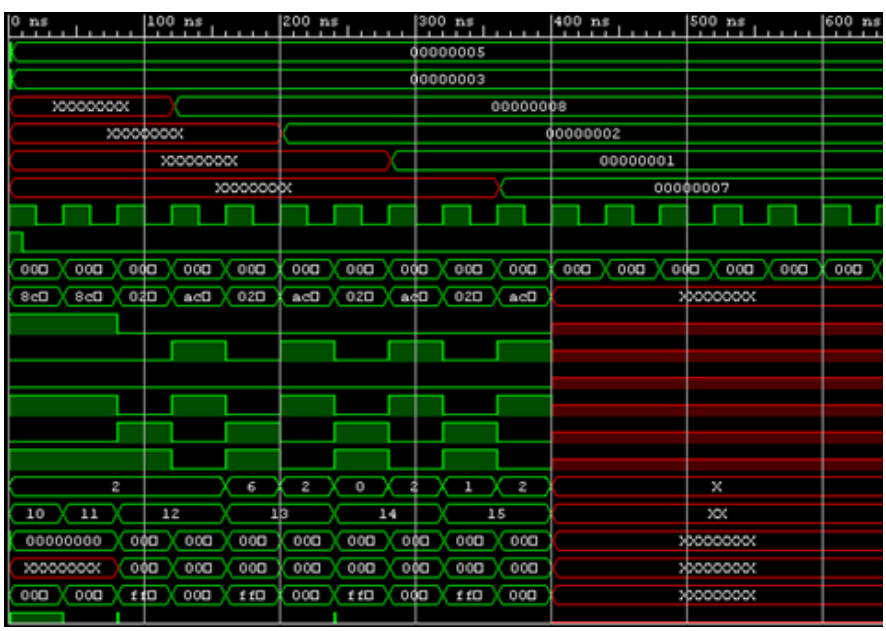
Lw $s0, 0($0)
Lw $s1, 4($0)
Add $s2, $s1, $s2
Sw $s2, 8($0)
sub $s3, $s1, $s2
Sw $s3, 12($0)
and $s4, $s1, $s2
Sw $s4, 16($0)
or $s5, $s1, $s2
Sw $s5, 20($0)

The code performs 4 operations on the numbers 5 and 3 which are stored in addresses 0x00 and 0x04 respectively. The operations are addition, subtraction, and-ing and or-ing. The results are stored in addresses 0x08, 0x0c, 0x10, 0x14 which should be 8, 2, 1, 7 respectively. The clock cycle time is assumed to be 25 ns in single cycle and assuming equal time stages therefore, clock cycle time is 5 ns in multi and pipeline.

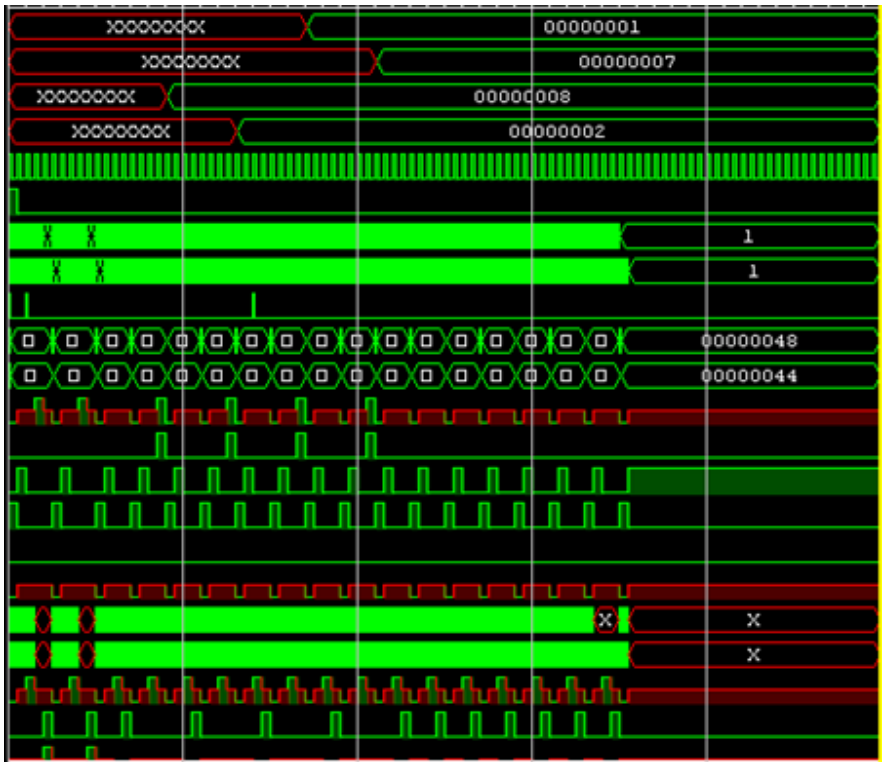
	Single Cycle	**Multi-cycle**	**Pipelining**
No. of cycles	10	42	14 + 1 stall = 15
CPI	1	4.2	1.5
Timing	25 x 10 = 250 ns	42 x 5 = 210 ns	15 x 5 = 75 ns

Observing the results, we can see that the multi-cycle has a slight speedup in performance while pipeline has significant speedup.

Single Cycle Simulation:



Multi-Cycle Simulation:



Pipeline Simulation:

Single Cycle	Multi-Cycle	Pipeline
Each instruction takes one clock cycle	Each instruction takes multiple clock cycles	Each instruction takes multiple clock cycles
There is one set of control signals for each instruction	There is one set of control signals for each cycle of each instruction	There is a set of control signals for each stage in the process
The clock period is the length of the slowest instruction	The clock period is the length of the slowest stage	The clock period is independent of the instruction
Extra registers are not required.	Extra registers are required to hold the result of one step for use in the next step.	Extra registers are required to allow simultaneous execution.
There is duplicate hardware (ALU), because we can use a functional unit for at most one subtask per instruction.	There is no duplicate hardware (ALU), because the instructions generally are broken into single FU steps	There **is** duplicate hardware (ALU - Registers), because we can use a functional unit for at most one subtask per instruction.

CONCLUSION:

Verilog is used for the implementation as it's the most straightforward method in addition to the ease of verification of the design and providing an organized results layout. Single-cycle MIPS is not efficient as Multi-cycle MIPS in terms of performance, hardware cost, and execution speed. The multi-cycle processor addresses these weaknesses by dividing instruction into 5 stages in different clock cycles which prevents any waste in the clock cycle period leading to faster execution speed than single-cycle MIPS, also the functional units can be used more than once to save hardware cost.

The single-cycle MIPS processor has two main drawbacks. Firstly, the clock cycle is required to last as long as the slowest instruction (LW), in contrast, most instructions are faster. Second, each functional unit is not required to be used more than once per instruction. So, the single-cycle MIPS processor design includes two adders and two ALU, and another separate instruction and data memory, which does not cost nor time-efficient.

On the other hand, the multicycle processor turns around those drawbacks by breaking every instruction execution into a chain of simpler steps each of one clock cycle corresponding to the functional unit operations that are needed. This allows instructions to take a different number of clock cycles and functional units to be shared within the execution of a single instruction. Therefore, compared to the single-cycle implementation the multicycle implementation of a processor uses fewer design modules and a faster clock speed.

REFERENCES:

1. D. K. Dennis et al., “Single cycle RISC-V micro architecture processor and its FPGA prototype,” in 2017 7th International Symposium on Embedded Computing and System Design, ISED 2017, 2018, doi: 10.1109/ISED.2017.8303926.

2. T. Subhashini, M. Kamaraju, and K. Babulu, “Single cycle risc micro architecture processor using clock gating technique,” Int. J. Sci. Technol. Res., 2019.

3. R. Uma, “Design and Performance Analysis of 8-bit RISC Processor using Xilinx Tool,” Int. J. Eng. Res. Appl.

4. H. S. M. Safaa S. Omran, “HARDWARE MODELLING OF A 32-BIT, SINGLE CYCLE RISC PROCESSOR USING VHDL,” ICIT 2013 6th Int. Conf. Inf. Technol., 2013.

5. S. Capkun and F. K. Gürkaynak, Multi-cycle MIPS Processor. http://www.syssec.ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/system-security-group-dam/education/Digitaltechnik_14/21_Architecture_MultiCycle.pdf

6. S. L. Harris and D. M. Harris, Digital Design and Computer Architecture. MA:

Elsevier, 2016.